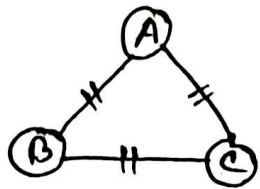


Graph

Different types of graph



Graph - G

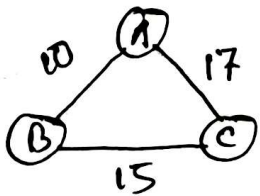
$$V = \text{vertices} = \{A, B, C\}$$

Vertex

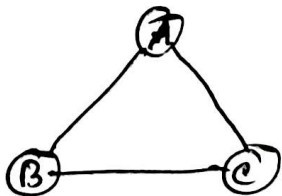
$$E = \text{Edges} = \{AB, AC, BC\}$$

BA CA CB

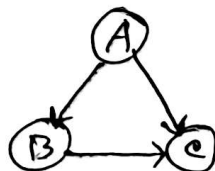
$$G \equiv \{V, E\}$$



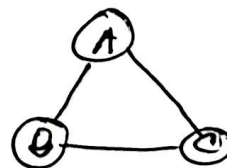
Weighted graph



Unweighted graph

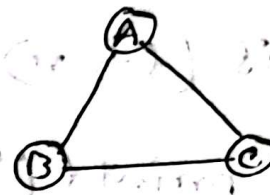


Directed graph



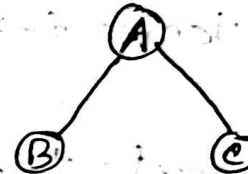
Undirected graph

$$E = \{AB, BC, AC\}$$



Cyclic graph

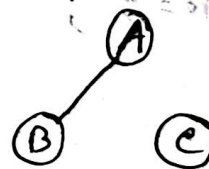
A-B-C-A
vertex repetition



Acyclic graph

(B-A-C)

(No cycle)

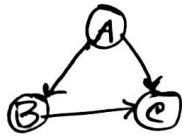


Disconnected graph

C Island

(No route to go vertex c)

Adjacency structure or Neighborhood Relation graph representation



- 2D Array

- Linked List

2D Array

	A	B	C
A	0	1	1
B	0	0	1
C	0	0	0

Number of
Vertices = $3 = |V|$
 $= n$

$$\text{Space complexity} = 9 = 3^2$$

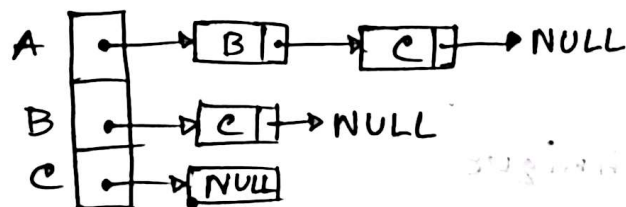
$$= V^2$$

$$= (|V|)^2$$

$$= O(|V|^2)$$

$$= O(n^2)$$

Linked List



$$\text{Space complexity} = 3 + 3$$

$$= |V| + |E|$$

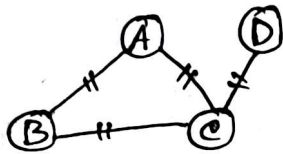
$$= O(|V| + |E|)$$

No. of
Vertices

No. of
Edges

→ out of syllabus

Degree of a Vertex



Degree of A $\equiv 2$

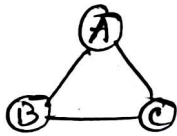
" of B $\equiv 2$

" of C $\equiv 3$

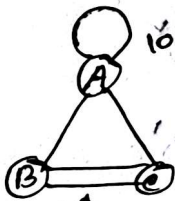
" of D $\equiv 1$

Total $\equiv 8 \equiv 2 \times 4$

degree = $2 \times \text{Total Edges}$



simple graph



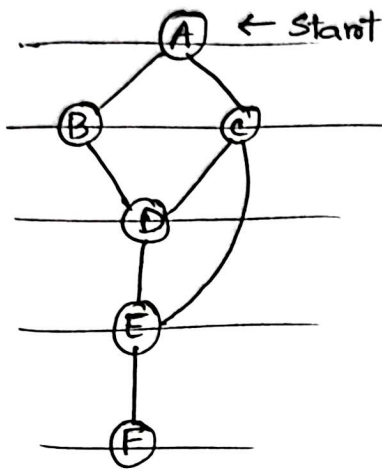
parallel edge

Multi graph

Graph traversal technique

- BFS (Breadth First search)
- DFS (Depth First search)

BFS



Queue

A				
---	--	--	--	--

BFS sequence $\equiv$ ABCEDEF

↓
Undeepst node first
shallowest

$v \leftarrow \text{Queue}$	visit v	visited seq $\leftarrow v$	Non-visited Adjacent u of v	$\text{Queue} \leftarrow u$					
$v = 'A'$	A	visited seq $\leftarrow A$	$u = \{B, C\}$	<table border="1"><tr><td>B</td><td>C</td><td></td><td></td><td></td></tr></table>	B	C			
B	C								
$v = 'B'$	B	visited seq $\leftarrow AB$	$u = \{A, D\}$	<table border="1"><tr><td>A</td><td>D</td><td></td><td></td><td></td></tr></table>	A	D			
A	D								
$v = 'C'$	C	visited seq $\leftarrow ABC$	$u = \{A, D, E\}$	<table border="1"><tr><td>D</td><td>D</td><td>E</td><td></td><td></td></tr></table>	D	D	E		
D	D	E							
$v = 'D'$	D	" " $\leftarrow ABCD$	$u = \{B, C, E\}$	<table border="1"><tr><td>D</td><td>E</td><td>E</td><td></td><td></td></tr></table>	D	E	E		
D	E	E							
$v = 'D'$	X	" " $\leftarrow X$	X	X					
$v = 'E'$	E	" " $\leftarrow ABCEDE$	$u = \{A, D, F\}$	<table border="1"><tr><td>E</td><td>F</td><td></td><td></td><td></td></tr></table>	E	F			
E	F								
$v = 'E'$	X	X	X	X					
$v = 'F'$	F	" " $\leftarrow \text{ABCEDEF}$	$u = \{F\}$	X					

Algorithm

Queue $\leftarrow$ Start

while (Queue $\neq$ Empty)

$V \leftarrow$ Queue

if (V is not visited)

visit V

visited sequence $\leftarrow V$

for all non-visited adjacents

u of V

Queue $\leftarrow u$

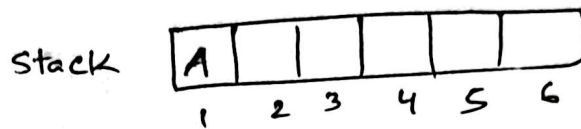
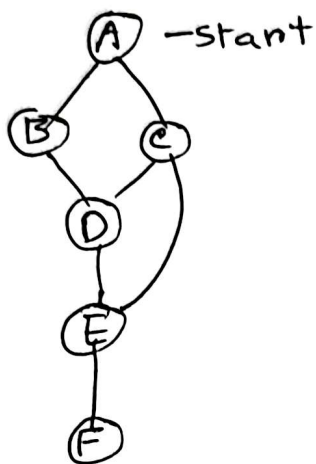
end for

End if

End while



DFS (Depth first search)



Stack ← start

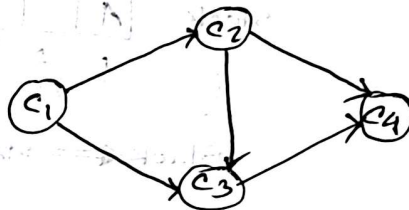
Stack	visit v	visited seq ← v	non visited adjacents u of v	stack ← u					
A	A	A	$u = \{B, C\}$	<table border="1"><tr><td>B</td><td>C</td><td></td><td></td><td></td></tr></table>	B	C			
B	C								
	C	AC	$u = \{A, D, E\}$	<table border="1"><tr><td>B</td><td>D</td><td>E</td><td></td><td></td></tr></table>	B	D	E		
B	D	E							
	E	ACE	$u = \{D, F\}$	<table border="1"><tr><td>B</td><td>D</td><td>F</td><td></td><td></td></tr></table>	B	D	F		
B	D	F							
	F	ACEF	$u = \{ \}$	X					
	D	ACEFD	$u = \{B, C, E\}$	<table border="1"><tr><td>B</td><td>C</td><td>E</td><td></td><td></td></tr></table>	B	C	E		
B	C	E							
	B	ACEFDB	$u = \{A, D\}$	X					
	X	X	X	X					
	X	X	X	X					

∴ DFS seq = ACEFDB

Algorithm same as
BFS

Topological Ordering

Ics
|
SPL
|
DSA-I
|
OOP
|
DSA-II



c1 c2 c3 c4

DAG (Directed Acyclic Graph)

c1 c2 c3 $\cancel{c4}$
DAG

c2 c3 c4 $\cancel{c1}$
DAG

c1 c2 c4 c3
DAG

$\therefore$ Given graph is directed acyclic graph.

order $\Leftarrow \{ \}$

Queue $\Leftarrow \{ c1 \}$

m $\Leftarrow$ Queue order $\Leftarrow$ m

m = 'c1'

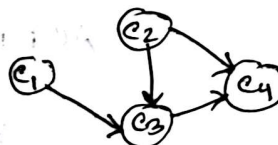
order $\Leftarrow$ c1

mn

Disconnect mn from graph

Queue $\Leftarrow$ n

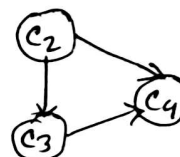
c1 c2
m n
~~c3~~



Queue = $\{ c1 \}$

n shouldn't have any incoming edge

c1 c3
 $\downarrow$ $\downarrow$
3 3

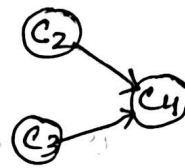


X

Graph can't be disconnected

$m = 'c_2'$ order $\leftarrow c_1 c_2$ $m \ n$
 $c_2 \ c_3$

Agar supaya bisa masuk



$Queue = \{c_3\}$



$m = 'c_3'$ order $\leftarrow c_1 c_2 c_3$ $c_3 \ c_4$

bisa masuk



Empty

$Queue = \{c_4\}$

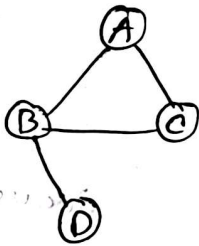
$m = 'c_4'$ order $\leftarrow c_1 c_2 c_3 c_4$ X



Tree Algorithm

Tree

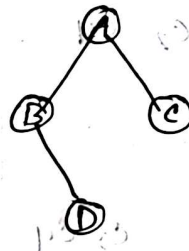
A tree is a connected acyclic graph



Connected ✓

cycle (A-B-C-A)

acyclic ✗

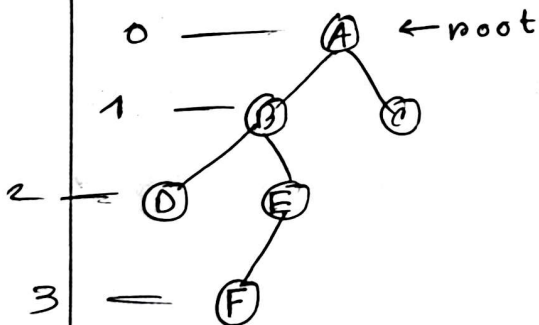


Tree

connected ✓

acyclic ✓

Family structure of tree



A has no father

B, C siblings
D, E siblings

A has two children: B, C (2 children)

B " " " : D, E (2 ")

E " " " : F (1 ")

C " " " : (0 ")

F " " " : (0 ")

max children = 2

∴ Binary tree

A level = 0

B, C " = 1

D, E " = 2

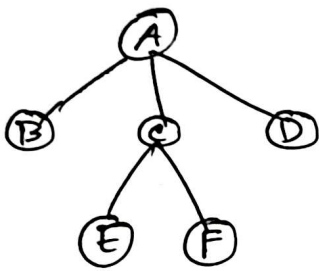
F " = 3

height = 3

A is grand father
of D, E

A is Ancestor
of F

Descendants of
A → B, C, D, E, F



$A \rightarrow B, C, D$

B, D have no child

$C \rightarrow E, F$

Ternary tree

height = max (level i)

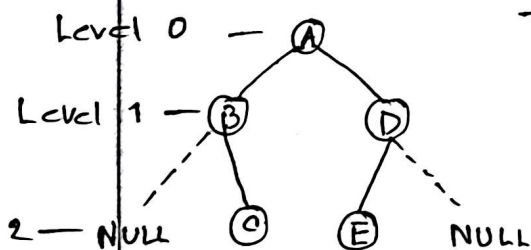
Data structure for Binary tree

- Array
- Linked list

Array

tree

A	B	D	-	C	E	-
1	2	3	4	5	6	7



Index of B using $A = 2 * \text{father index}$

$$= 2 * 1$$

$$= 2$$

Binary tree

Index of D = $2 * \text{father index} + 1$

$$= 2 * 1 + 1$$

$$= 3$$

Index of C = $2 * \text{father index} + 1$

$$= 2 * 2 + 1$$

$$= 4 + 1$$

$$= 5$$

father
known.

child = ?

$$\text{Index of B} = \left\lfloor \frac{\text{child index}}{2} \right\rfloor$$

$$= \left\lfloor \frac{\text{index of C}}{2} \right\rfloor$$

$$= \left\lfloor \frac{5}{2} \right\rfloor$$

$$= \lfloor 2.5 \rfloor$$

$$= 2$$

child known

father = ?

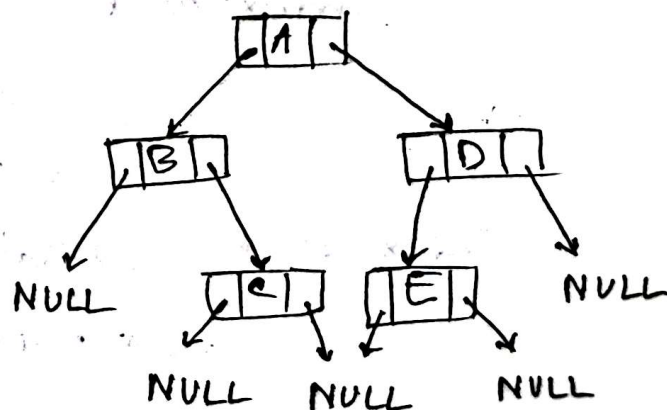
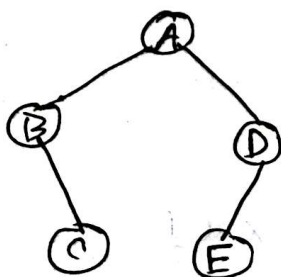
$$\text{Index of D} = \left\lfloor \frac{\text{child index}}{2} \right\rfloor$$

$$= \left\lfloor \frac{\text{index of E}}{2} \right\rfloor$$

$$= \left\lfloor \frac{6}{2} \right\rfloor$$

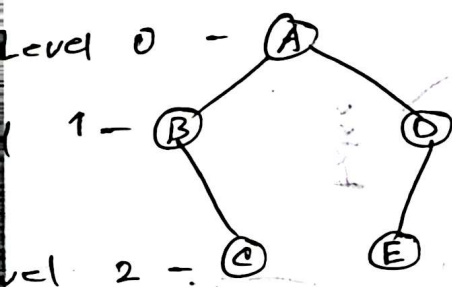
$$= 3$$

Linked list



Tree traversal Algorithms:

- Preorder (root - left - right)
- In order (left - root - right)
- Postorder (left - right - root)
- Level order (Level by level)



preorder: A B C D E

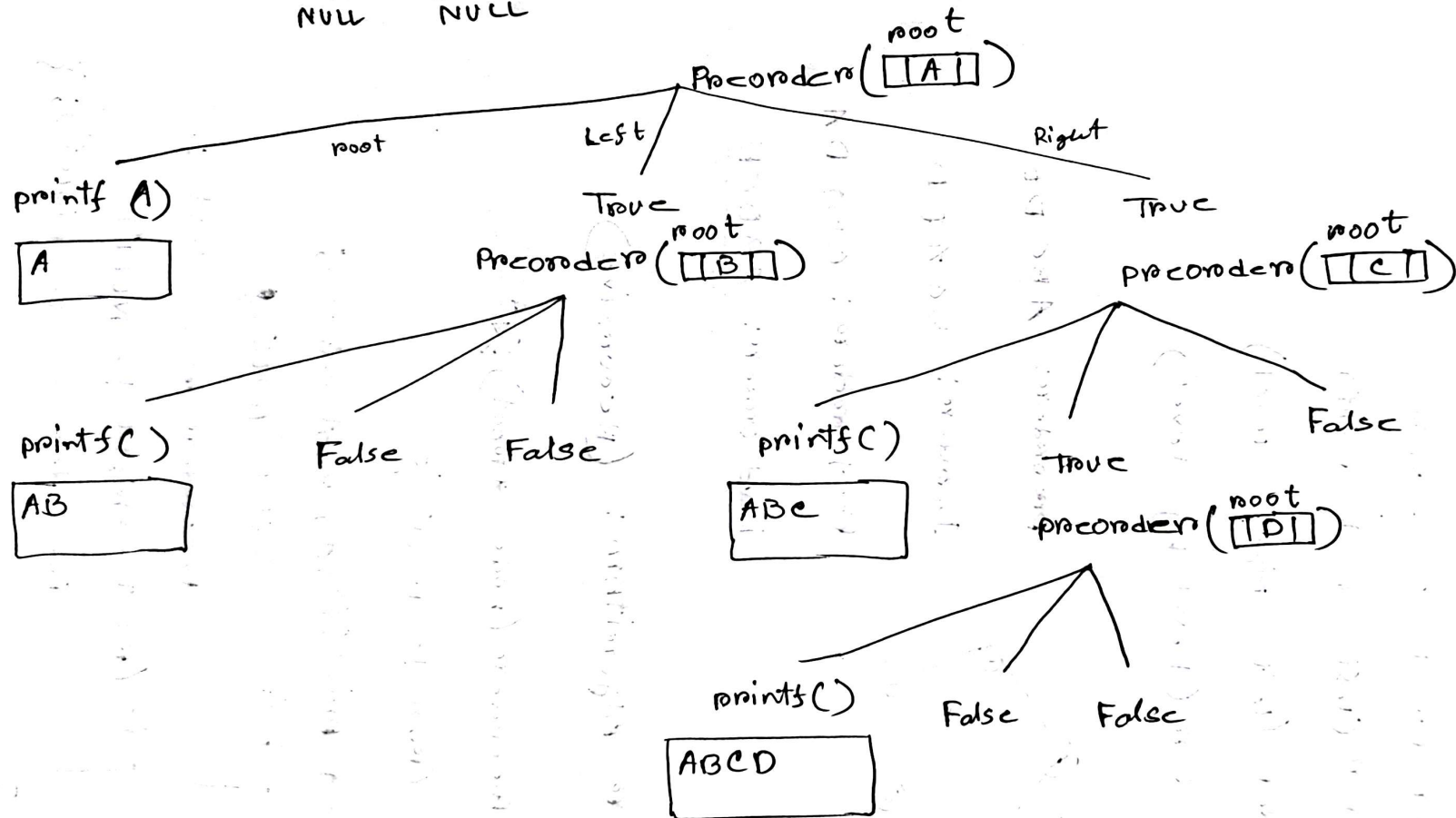
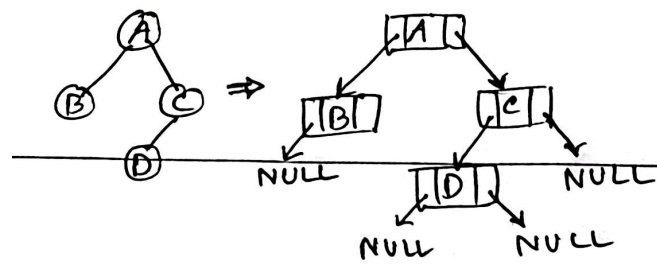
In order: B C A E D

Post order: C B E D A

Level order: $\begin{matrix} A & B & D & C & E \\ 0 & 1 & 2 \end{matrix}$

Preorder Algorithm: (Recursive)

```
void Preorder (node *root) {  
    if (root != NULL) {  
        root → printf("%c", root → into);  
        left { if (root → left != NULL) {  
                Preorder (root → left);  
            }  
        right { if (root → right != NULL) {  
                Preorder (root → right);  
            }  
    }  
}
```



```
Void Preorder (node *root) {
```

```
    if (root != NULL) {
```

```
        root → printf ("%c", root → info);
```

```
        left → { if (root → left != NULL) {
                    Preorder (root → left);
```

```
                if (root → right != NULL) {
                    Preorder (root → right);
```

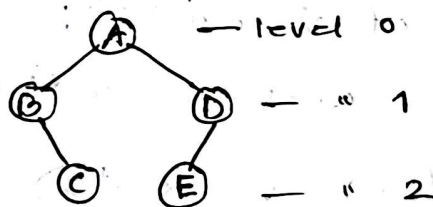
```
                }
```

```
    }
```

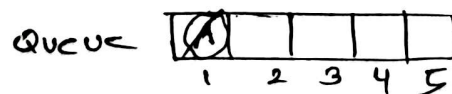
{ Inorder
post order }

Same

Level order:



ABDCE



$V \leftarrow \text{Queue}$

visit V

$\text{levelorder} \leftarrow V$

if left of V exist

$\text{Queue} \leftarrow \text{left}(V)$

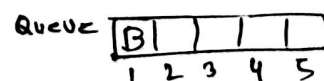
if right of V

exist $\text{Queue} \leftarrow \text{right}(V)$

$V = A$

A

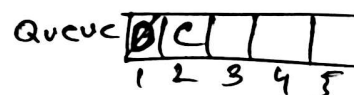
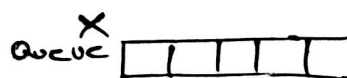
level order = A



$V = B$

B

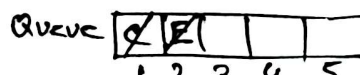
" " = AB



$V = D$

D

" " = ABD



X

$v=c$ C " " = ABDC X X

$v=E$ E " " = ABDCE X X

level order

Algorithm

Queue $\leftarrow$ root

while (Queue $\neq$ Empty) {

$v \leftarrow$ Queue

visit v

Level order $\leftarrow v$

if (left of v exists);

Queue $\leftarrow$ left(v)

if (right of v exists)

Queue $\leftarrow$ right(v)

}

return output Levelorder;

Ques: Construct a unique binary tree from the following inorder and postorder sequences.

(left/right)

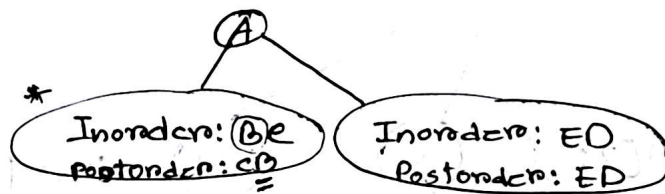
Inorder: BCAED (left-root-right) * must be given

(root) Postorder: CBEDA (left-right-root)

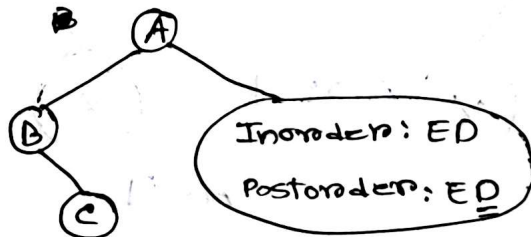
root

A

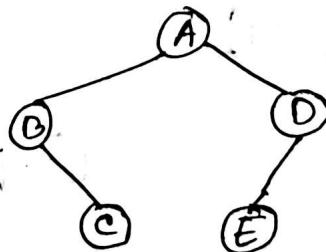
Tree



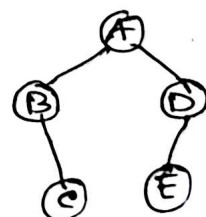
B



D



Binary tree:



Preorder:
Inorder:

Binary tree

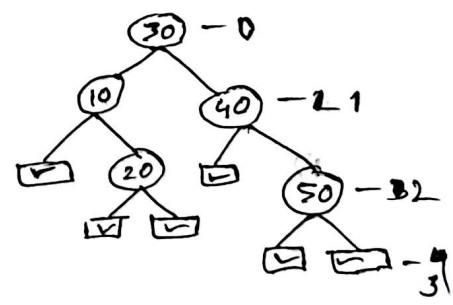
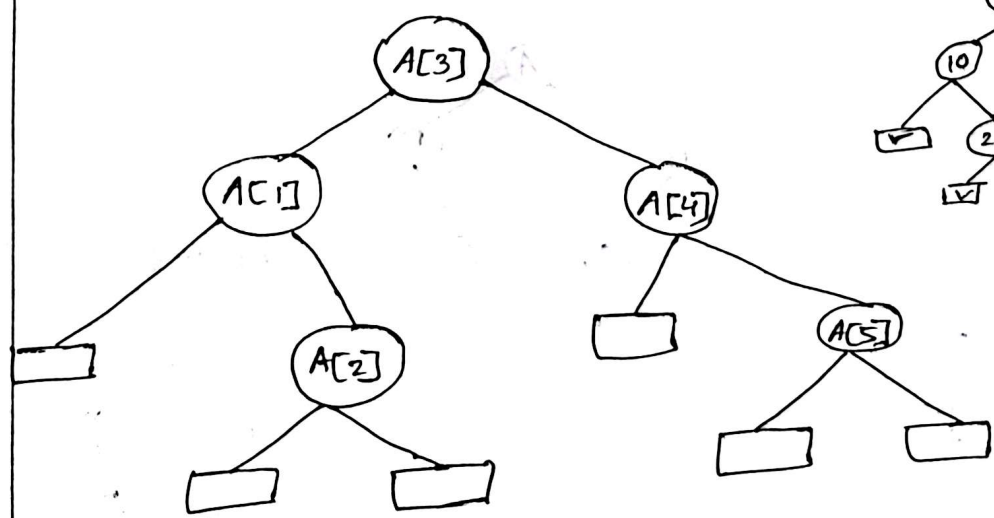
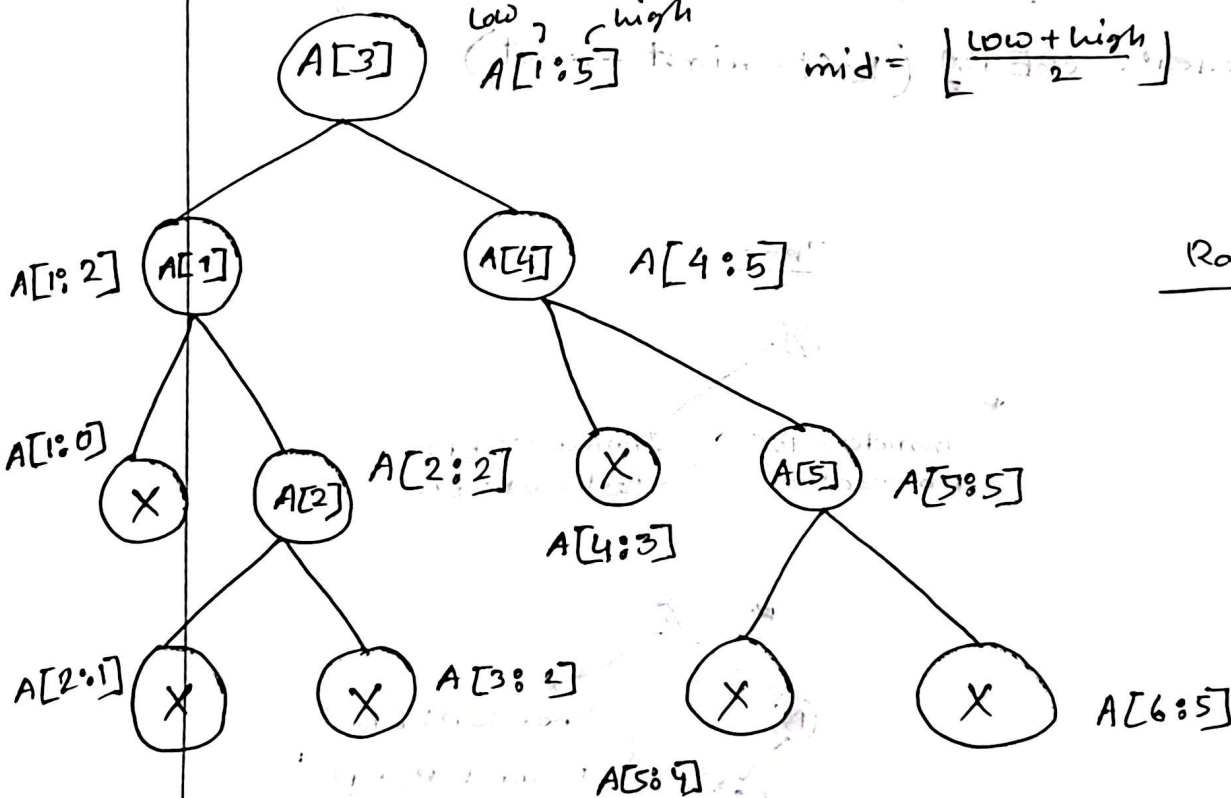
Binary search Tree (BST)

Data:

10	20	30	40	50
1	2	3	4	5

low, high
 $A[1:5]$ mid = $\left\lfloor \frac{low + high}{2} \right\rfloor$

Rain gold



Binary search tree

- Binary search tree properties.

○ = circular node = internal node = $I(n)$

□ = square node = External node = $E(n)$

Circular, $I(n) = 5$

square, $E(n) = 6$

$$\begin{aligned} E(n) &= I(n) + 1 \\ \Rightarrow 6 &= 5 + 1 \end{aligned}$$

External path length,

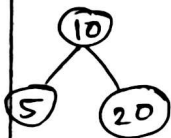
$$E(T) = 2 + 2 + 3 + 3 + 3 + 3$$

$$= 16$$

Internal path length,

$$I(T) = 0 + 1 + 1 + 2 + 2$$

$$= 6$$



root $\geq$ left-child
root $<$ right-child
properties of
Binary Search Tree

$$\begin{aligned} E(T) - I(T) &= 2 * n \\ \Rightarrow 16 - 6 &= 2 * 5 \end{aligned}$$

$$\Rightarrow 10 = 10$$

→ circular node

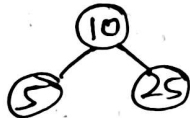
10:

10

5:

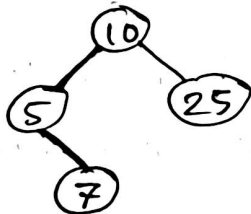


25:

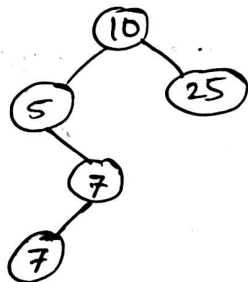


7:

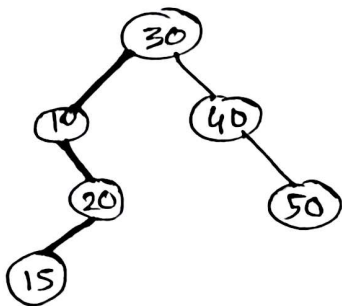
7



7:



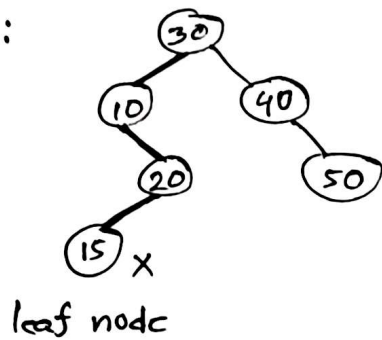
Insertion in a binary search tree:



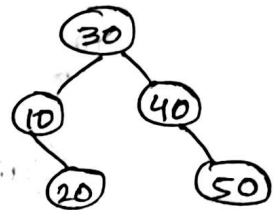
insert(15)

Deletion from a BST:

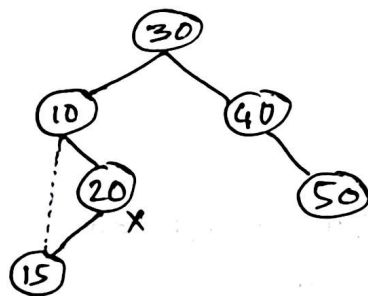
Case 1:



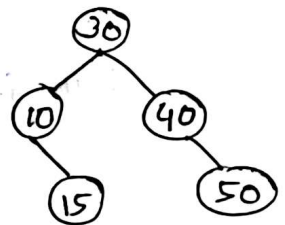
Delete(15)
leaf node



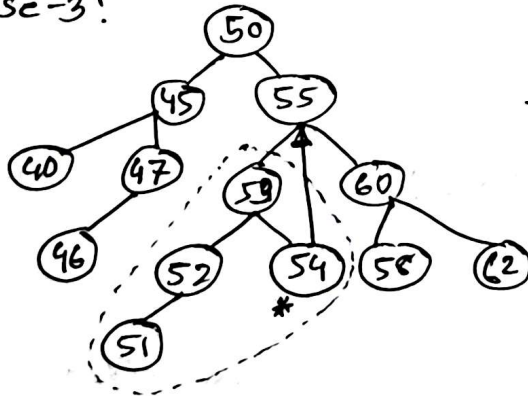
Case-2:



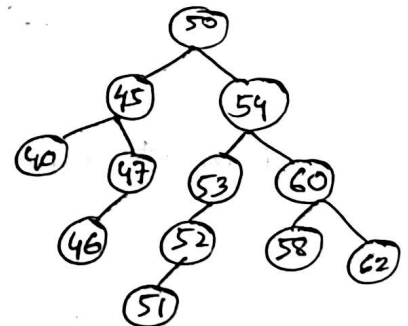
Delete(20)
One child node



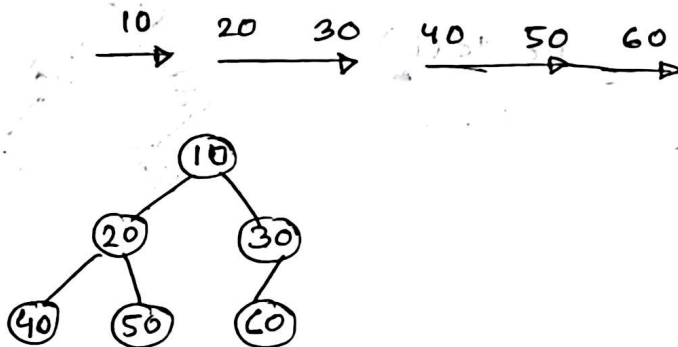
Case-3:



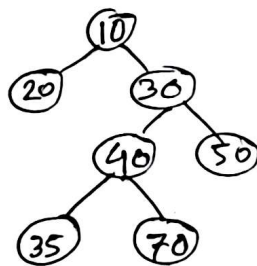
Delete(55)



Complete binary Tree

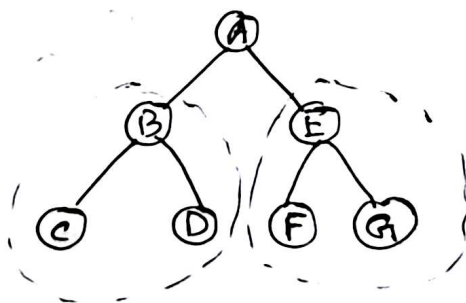


Full form binary Tree :

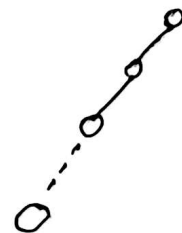


or, Full completed binary Tree.

Perfect/Balanced Binary Tree:

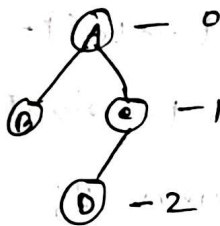


One sided Binary Tree (Acentric)



Binary Tree Height calculation:

$$\text{height}(T) = \max(\text{height}(T_L), \text{height}(T_R)) + 1$$



Method 1: $\text{height} = \max(\text{level})$
 $= 2$

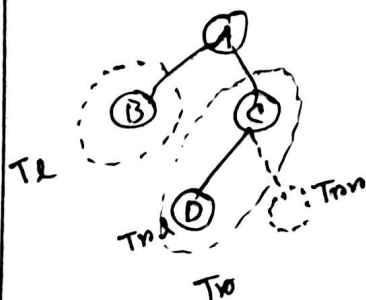
Method 2: Recursive formula

$$\text{height}(T) = \max(\text{height}(T_L), \text{height}(T_R)) + 1$$

Tree = NULL height = -1

Tree = height = 0
 NULL NULL

Otherwise, Recursive formula



$$\text{height}(T) = \max(\text{height}(T_L), \underline{\text{height}(T_R)}) + 1$$

$$= \max(0, \underline{\max(\text{height}(T_{L2}), \text{height}(T_{R2})) + 1}) + 1$$

$$= \max(0, \max(0, -1) + 1) + 1$$

$$= \max(0, 0 + 1) + 1$$

$$= \max(0, 1) + 1$$

$$= 1 + 1$$

$$= 2$$

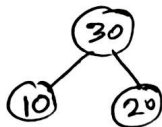
```
int height(node *root) {  
    if (root == NULL)  
        return -1;  
  
    else if (root->left == NULL)  
        || (root->right == NULL)  
        return 0;  
  
    else  
        return max(height(root->left), height(  
            root->right)) + 1;  
}
```

```
int max(int x, int y) {  
    if x > y  
        return x;  
    else return y;  
}
```

Heap Sort

Heapify :

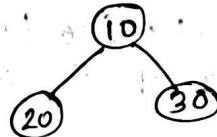
max heap tree



father $\geq$ child

(Ascending order)

min heap tree



father $\leq$ child

(Descending)

- Heapify

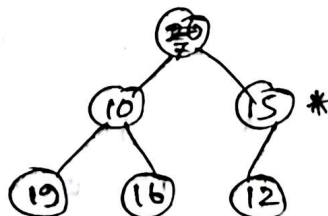
- Heap sort

Sort the following data into ascending order using heap sort Algorithm.

A	20	7	10	15	12	16	12
	1	2	3	4	5	6	

No. of elements, $n = 6$

Heapify/Adjust (max)



$$i = \lfloor \frac{n}{2} \rfloor$$

$$= \lfloor \frac{6}{2} \rfloor$$

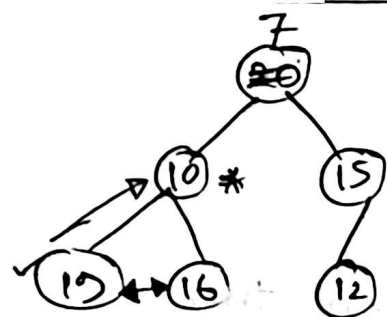
$$= 3$$

complete

Binary

Tree

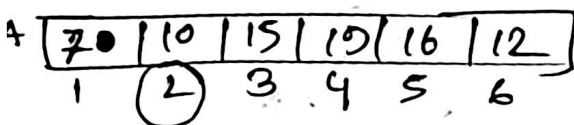
A	20	7	10	15	12	16	12
	1	2	3	4	5	6	



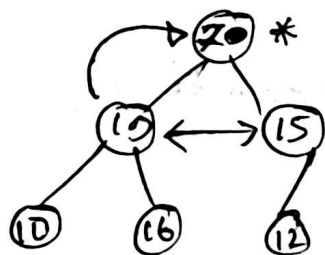
$$i = i - 1$$

$$= 3 - 1$$

$$= 2$$

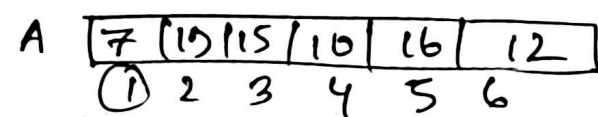


item = 7



$$i = 2 - 1$$

$$= 1$$

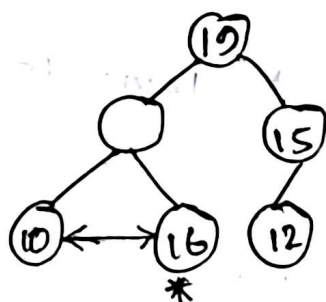


Left right compare (range)
16

compare 16 with item = 7

$$16 > 7$$

Then proceed.



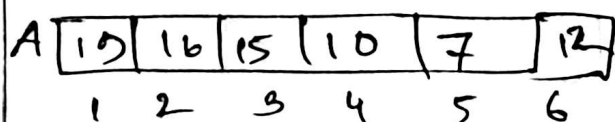
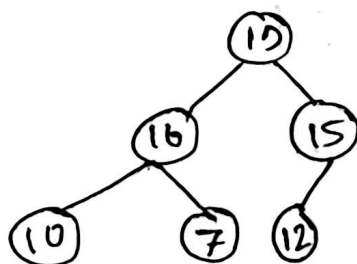
* Left/Right child compare
(10, 16)

$$\text{Largest} = 10$$

* Largest > 10 $A[i]$

$$10 > 10 - \text{True}$$

Push 10 to upper level



i) compare (10, 15)

ii) largest = 15

iii) largest > 7 - True

push (15)

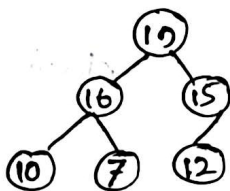
i) compare (10, 16)

ii) largest = 16

iii) largest > 7 True

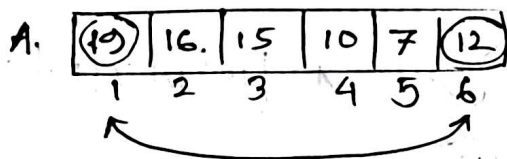
push (16)

Step 2: Apply heap sort Algorithm on max heap tree.



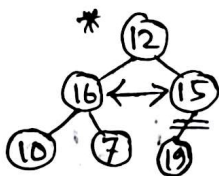
$i = n = 6$

$A[i] \leftrightarrow A[i]$

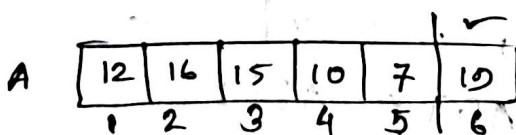


$A[i] \leftrightarrow A[6]$

$19 \leftrightarrow 12$



again
max heap build



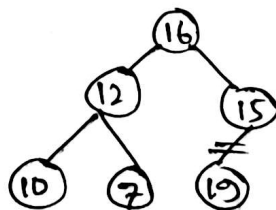
Adjust (12)

i) compare (16, 15)

largest = 16

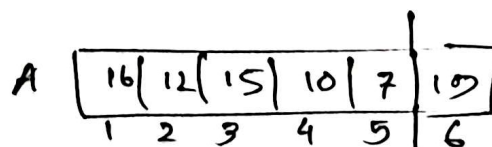
$16 > 12$ - True

push (16)



ii) 10 is largest

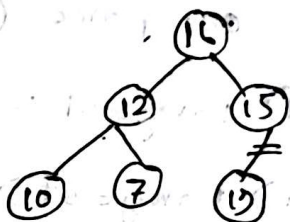
$10 > 12$ False



$$i = i - 1$$

$$= 5 - 1$$

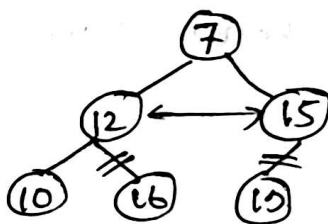
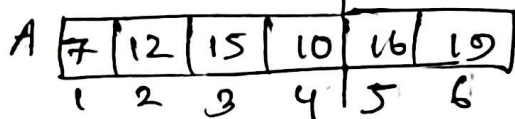
$$= 5$$



$$A[i] \leftrightarrow A[i] \text{ swap}$$

$$A[i] \leftrightarrow A[5]$$

$$16 \leftrightarrow 7$$



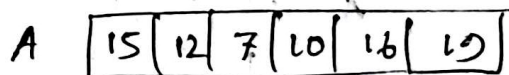
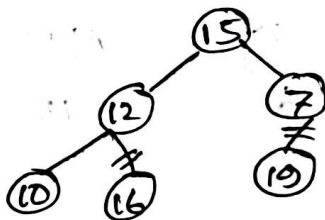
max heap build

Adjust $A[i] = 7$

① compare (12, 15)

largest = 15

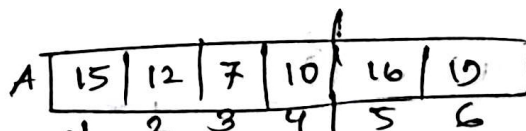
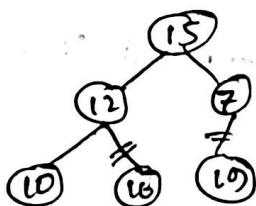
push (15)



$$i = i - 1$$

$$= 5 - 1$$

$$= 4$$



$$A[i] \leftrightarrow A[i]$$

$$A[i] \leftrightarrow A[4]$$

$$15 \leftrightarrow 10$$



Algorithm

Procedure heapsort(A, n)

Step 1 $\left[\begin{array}{l} \text{for } i = \lfloor \frac{n}{2} \rfloor \text{ down to } 1 \\ \quad \text{maxheapify}(A, i, n) \\ \text{end for} \end{array} \right.$

Step 2 $\left[\begin{array}{l} \text{for } i = n \text{ down to } 2 \\ \quad A[1] \leftrightarrow A[i] \\ \quad \text{maxheapify}(A, 1, i-1) \\ \text{end for} \end{array} \right.$

end procedure

Procedure
~~void~~ maxheapify(A, i, n)

item = $A[i]$

$j = 2i$

while ($j \leq n$)

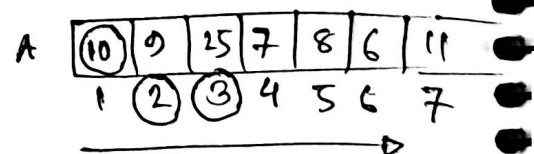
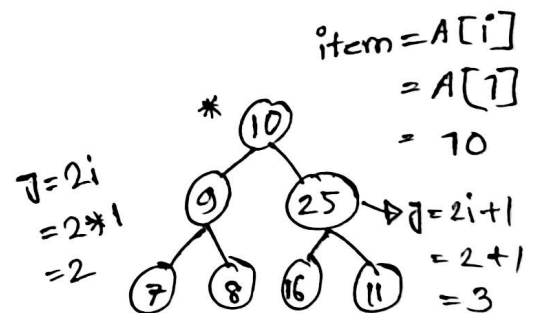
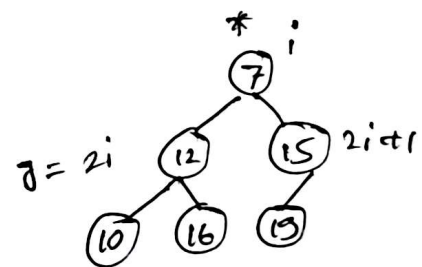
if $j < n$ and ($A[j] < A[j+1]$)

$j = j + 1$

if ($A[j] \leq \text{item}$) exit loop

$A[\frac{j}{2}] = A[j]$

$j = 2j$



No. of elements

$n = 7$

end while

$A[\frac{j}{2}] = \text{item}$

End procedure

$j = 2 \quad j \leftarrow n$

$j = 2 \quad j < n \quad \text{and} \quad A[j] < A[j+1]$

$= 2 < 7$

$= \text{True}$

$A[2] < A[3]$

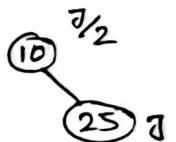
$= 9 < 25$

$= \text{True}$

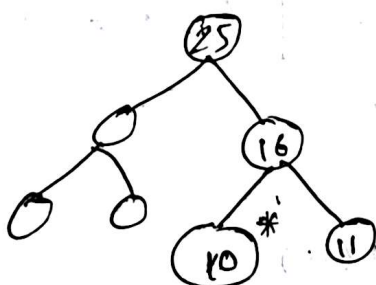
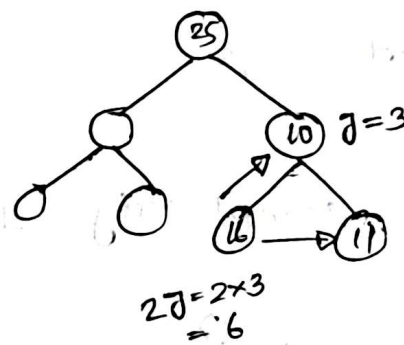
$\therefore j = j + 1$

$= 2 + 1$

$= 3$



relation
father-child



item = 10

$$\frac{j}{2} = \frac{12}{2} = 6$$

$A[6] = \text{item}$

$= 10$

$j = 6 \quad j \leq n \quad j < n \quad \text{and} \quad A[j] < A[j+1]$

$6 \leq 7$

T

$6 < 7$

T

$A[6] < A[7]$

$16 < 11$

False

$j = 2j$

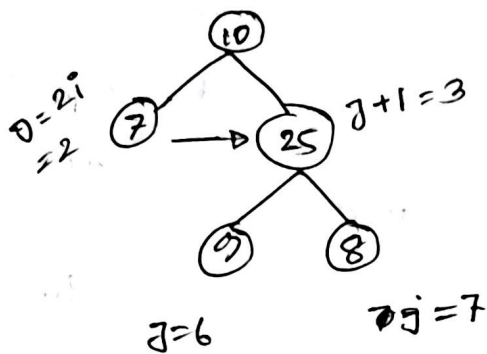
$= 2 * 6$

$= 12$

$j \leq n$

$12 \leq 7$

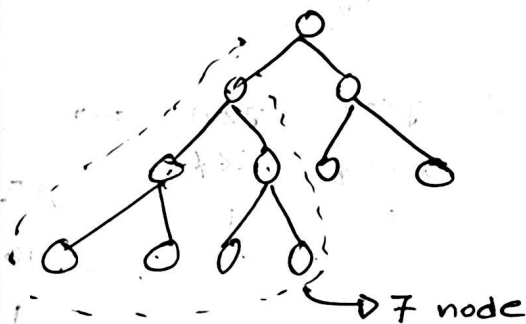
False



** in complete binary tree there should be maximum $\frac{2n}{3}$ node in any sub tree.

if $A[j] \leq item$
 ~~$A[6] \leq$~~
 $(9 \leq 10)$ True
 exit loop.

Time complexity Analysis: (max heapify)



$$n=11$$

$$\frac{2n}{3} = \left\lfloor \frac{2*11}{3} \right\rfloor \text{ (subtree)}$$

$$= 7$$

Recurrence relation for time, $T(n)$

$$T(n) = \begin{cases} 0 & n=1 \\ T\left(\frac{2n}{3}\right) + c & n > 1 \end{cases}$$

Adjust time

for comparison left/right

when $n > 1$

$$T(n) = T\left(\frac{2n}{3}\right) + c$$

$$T = T\left(\frac{2^r}{3^r} n\right) + c + c$$

$$= T\left(\frac{2^k}{3^k} n\right) + 2c$$

...

$$= T\left(\frac{2^k}{3^k} n\right) + kc$$

$$= T(1) + kc$$

$$\text{constant} = 0 + c \frac{\log_2 n}{\log_2 1.5}$$

$$= \frac{c}{\log_2 1.5} \log_2 n$$

$$= D \log_2 n$$

Heap sort time

$$\begin{cases} T(n) = 0 \\ n = 1 \end{cases}$$

$$T(n) = \frac{n}{2} * \text{max heapify time}$$

$$+ (n-2+1) * \text{max heapify time}$$

$$= \frac{n}{2} * \frac{c}{\log_2 1.5} \log_2 n + (n-1) * \frac{c}{\log_2 1.5} \log_2 n$$

$$= O(n \log_2 n)$$

$$n = \frac{2n}{3}$$

$$T\left(\frac{2n}{3}\right) = T\left(\frac{2}{3} * \frac{2n}{3}\right) + c$$

$$= T\left(\frac{2^r}{3^r} n\right) + c$$

$$\frac{2^k}{3^k} n = 1$$

$$\Rightarrow n = \left(\frac{3}{2}\right)^k$$

$$\Rightarrow \log_2 n = \log_2 \left(\frac{3}{2}\right)^k$$

$$\Rightarrow \log_2 n = k \log_2 1.5$$

$$\Rightarrow k = \frac{\log_2 n}{\log_2 1.5}$$

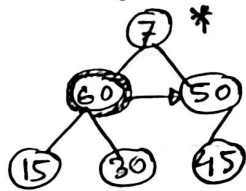
$$T(n) = 0 \text{ when } n = 1$$

$$\Rightarrow T(1) = 0$$

$$\begin{aligned} & \left(\frac{n}{2} - 1 + 1\right) D \log_2 n + (n-2+1) [c + D \log_2 n] \\ &= \frac{n}{2} D \log_2 n + (n-1) (c + D \log_2 n) \end{aligned}$$

Heapify

manual tracing



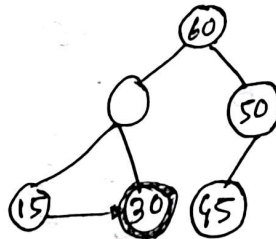
No. of elements, $n=6$

A	7	60	50	15	30	45
	1	2	3	4	5	6

item = $A[i] = A[1] = 7$, $j = 2 * i$
 $= 2 * 1$
 $= 2$

$j \leq n$
 $2 \leq 6$ True

if $j < n$ & $A[j] < A[j+1]$
 $2 < 6$ & $A[2] < A[3]$
 True $60 < 50 = \text{False}$



$j = j + 1$
 X

is $A[j] \leq \text{item}$
 $A[2] < 7$
 $60 < 7$ False
 exit loop X

$A[\lfloor \frac{j}{2} \rfloor] = A[j]$
 $\Rightarrow A[\lfloor \frac{2}{2} \rfloor] = A[2]$
 $\Rightarrow A[1] = A[2]$
 $= 60$

A	60		50	15	30	45
	1	2	3	4	5	6

$j = 2 * j$
 $= 2 * 2$
 $= 4$

$4 \leq 6$ True

if $4 < 6$ & $A[4] < A[5]$

True

$15 < 30$ True

$j = j + 1$
 $= 4 + 1$
 $= 5$

if $A[5] \leq 7$

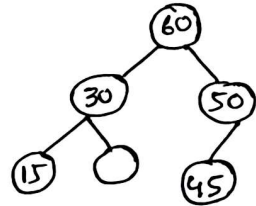
$30 \leq 7$

False

exit loop X

$A[\lfloor \frac{5}{2} \rfloor] = A[2]$

$\Rightarrow A[2] = A[5]$



A	60	30	50	15		45
	1	2	3	4	5	6

$j = 2 * j$

$= 2 * 5 = 10$

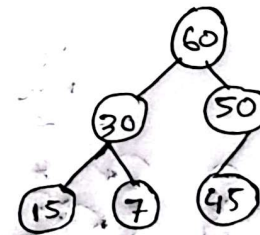
$10 \leq 6$ False

$A[\lfloor \frac{10}{2} \rfloor] = \text{item}$

$\Rightarrow A[\lfloor \frac{10}{2} \rfloor] = 7$

$\Rightarrow A[5] = 7$

A	60	30	50	15	7	45
	1	2	3	4	5	6



Time complexity comparison:

Selection = $O(n^2)$

= $O(n * n)$

Heap sort = $O(n * \log_2 n)$

$n > \log_2 n$

$\therefore$ Heap sort
take less time.

Graph

Tree

BST

Heap sort